

**ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KỸ THUẬT MÁY TÍNH**



ĐỒ ÁN 1

**THIẾT KẾ PHẦN CỨNG TĂNG TỐC AUTOENCODER
KHỬ NHIỄU ẢNH TRÊN FPGA**

Hardware Acceleration of Denoising Autoencoder on FPGA (KV260)

Giảng viên hướng dẫn: TS. Phạm Hoài Luân

Sinh viên thực hiện:

Lương Ngọc Đại - 23520215

Tp. Hồ Chí Minh, 2026

MỤC LỤC

Chương 1. GIỚI THIỆU ĐỀ TÀI	7
1.1. Lý do chọn đề tài.....	7
1.2. Mục tiêu đề tài	7
Chương 2. CƠ SỞ LÝ THUYẾT	8
2.1. Mạng Autoencoder Tích chập (CNN Autoencoder).....	8
2.1.1. Khái niệm Autoencoder.....	8
2.1.2. Mạng khử nhiễu cổ điển (Denoising Autoencoder - DAE).....	8
2.1.3. Convolutional Autoencoder (CAE).....	9
2.1.4. Phép toán tích chập (Convolution).....	9
2.1.5. Hàm kích hoạt	10
2.1.6. MaxPooling.....	10
2.1.7. UpSampling	10
2.1.8. Kiến trúc Encoder – Decoder trong đồ án.....	10
2.2. Tổng quan về công nghệ FPGA KV260	10
2.2.1. Kiến trúc Zynq UltraScale+ MPSoC.....	10
2.2.2. Ưu điểm FPGA trong triển khai neural network	11
2.3. High-Level Synthesis (HLS)	11
2.3.1. Khái niệm HLS	11
2.3.2. Kiểu dữ liệu Fixed-point ap_fixed	11
2.3.3. Các pragma tối ưu hóa HLS	12
2.3.4. Chuẩn giao tiếp AXI.....	12
2.4. Tập dữ liệu MNIST và bài toán khử nhiễu	13
2.4.1. Tập dữ liệu MNIST	13
2.4.2. Bài toán khử nhiễu ảnh.....	13
2.4.3. Quy trình tiền xử lý dữ liệu	13
Chương 3. TRIỂN KHAI ĐỀ TÀI	14
3.1. Thiết kế và lựa chọn kiến trúc mô hình Python.....	14
3.1.1. Môi trường và cấu hình huấn luyện	14
3.1.2. Hai phương án kiến trúc được đề xuất	14
3.1.3. So sánh kết quả huấn luyện hai mô hình	16
3.1.4. Phân tích và quyết định lựa chọn kiến trúc cho FPGA.....	16
3.2. Chuyển đổi mô hình sang C++ và tối ưu HLS	16
3.2.1. Quy trình chuyển đổi trọng số Python sang C++	17

3.2.2. Cấu trúc mã nguồn C++ HLS	17
3.2.3. Tối ưu hóa mô hình cho HLS	18
3.2.4. So sánh các cấu hình pragma tối ưu hóa	18
3.3. Tích hợp IP vào hệ thống SoC.....	20
3.3.1. Kiến trúc hệ thống SoC	20
3.3.2. Address Mapping	21
3.3.3. Thanh ghi AXI-Lite điều khiển.....	22
3.4. Phần mềm điều khiển trên ARM (KV260)	22
3.4.1. Phương pháp truy cập phần cứng.....	22
3.4.2. Cấu trúc dữ liệu trong BRAM	23
3.4.3. Luồng điều khiển IP	24
3.4.4. Quy trình triển khai trên KV260	25
Chương 4. KIỂM TRA VÀ ĐÁNH GIÁ KẾT QUẢ	25
4.1. Thiết lập môi trường thực nghiệm	25
4.2. Kết quả C Simulation và CoSim (Vitis HLS)	26
4.3. Kết quả thực nghiệm trên FPGA KV260	27
4.4. Đánh giá kết quả	27
4.4.1. Độ chính xác chuyển đổi Fixed-point	27
4.4.2. Chất lượng khử nhiễu	27
4.4.3. Hiệu năng.....	27
4.4.4. So sánh giữa C Simulation và FPGA	28
4.4.5. So sánh hiệu năng và tài nguyên với công trình liên quan.....	28
4.4.6. Tổng kết đánh giá.....	29
Chương 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	29
5.1. Kết luận	29
5.2. Hướng phát triển	30

DANH MỤC HÌNH ẢNH

Hình 1 Cấu trúc mã nguồn C++ HLS	17
Hình 2 Cấu hình tối ưu tài nguyên	19
Hình 3 Hệ thống SoC hoàn chỉnh.....	20
Hình 4 Địa chỉ AXI của hệ thống SoC	21
Hình 5 Ánh xạ bộ nhớ bằng mmap()	23
Hình 6 Cấu trúc dữ liệu trong BRAM	23
Hình 7 Lưu đồ điều khiển IP	24
Hình 8 Kết quả C Simulation	26
Hình 9 Kết quả C/RTL CoSimulation	27
Hình 10 Kết quả thực nghiệm FPGA KV260 - 100 ảnh	27

DANH MỤC BẢNG

Bảng 1 So sánh Dense DAE và Convolutional Autoencoder.....	9
Bảng 2 Kiến trúc mạng Denoising Autoencoder	10
Bảng 3 Kiến trúc mô hình nhỏ (665 tham số).....	14
Bảng 4 Kiến trúc mô hình lớn (~12.000 tham số).....	15
Bảng 5 So sánh kết quả huấn luyện Model nhỏ và Model lớn	16
Bảng 6 Phân tích đánh đổi và quyết định lựa chọn kiến trúc	16
Bảng 7 Các file header chứa trọng số Fixed-point.....	17
Bảng 8 So sánh ba cấu hình pragma tối ưu hóa HLS.....	19
Bảng 9 Bảng thanh ghi AXI-Lite từ denoiser_top_control_s_axi.v	22
Bảng 10 So sánh kết quả C Simulation và FPGA thực tế	28
Bảng 11 Đối chiếu hiệu năng và tài nguyên phần cứng với Zhao et al. (2018)	28

TÓM TẮT ĐỒ ÁN

Đồ án trình bày quá trình thiết kế và triển khai bộ tăng tốc phần cứng cho mô hình Convolutional Denoising Autoencoder nhằm khử nhiễu ảnh chữ số viết tay MNIST trên nền tảng FPGA AMD Xilinx Kria KV260 bằng phương pháp High-Level Synthesis (HLS). Mô hình được xây dựng và huấn luyện bằng Python, sau đó chuyển đổi sang C++ HLS với kiểu dữ liệu `ap_fixed` để phù hợp cho việc triển khai trên phần cứng.

Quy trình thực hiện bao gồm xây dựng và đánh giá kiến trúc mô hình, chuyển đổi trọng số sang định dạng fixed-point, tối ưu mã HLS, tổng hợp IP Core bằng Vitis HLS, tích hợp vào hệ thống SoC trên Vivado thông qua giao tiếp AXI và xây dựng chương trình điều khiển chạy trên bộ xử lý ARM của kit KV260 để thực hiện quá trình suy luận.

Kết quả thực nghiệm trên 100 ảnh kiểm thử cho thấy hệ thống đạt PSNR trung bình 42,97 dB so với mô hình Python và 19,11 dB so với ảnh gốc sau khử nhiễu, với độ trễ trung bình 5,01 ms/ảnh (tương đương 199,5 FPS). Sai lệch rất nhỏ giữa kết quả trên FPGA và mô hình Python chứng minh tính chính xác của quá trình chuyển đổi sang HLS và khả năng triển khai hiệu quả mô hình học sâu trên FPGA.

Chương 1. GIỚI THIỆU ĐỀ TÀI

1.1. Lý do chọn đề tài

Trong thời đại công nghiệp 4.0, các hệ thống xử lý ảnh thông minh ngày càng đóng vai trò quan trọng trong nhiều lĩnh vực như y tế, an ninh, tự động hóa và IoT. Tuy nhiên, việc triển khai các mạng học sâu trên các thiết bị nhúng đặt ra nhiều thách thức về năng lượng tiêu thụ, độ trễ xử lý và tài nguyên phần cứng.

Nền tảng FPGA (Field Programmable Gate Array) nổi lên như một giải pháp lý tưởng nhờ khả năng tính toán song song cao, tiêu thụ điện năng thấp và khả năng cấu hình lại linh hoạt. Đặc biệt, dòng kit KV260 của Xilinx tích hợp bộ xử lý ARM Cortex-A53 (PS) và ma trận logic lập trình (PL) trên cùng một chip, tạo ra kiến trúc SoC mạnh mẽ cho các ứng dụng AI nhúng.

Đề tài lựa chọn bài toán khử nhiễu ảnh chữ số viết tay (Denoising Autoencoder) vì đây là bài toán điển hình trong xử lý ảnh, đòi hỏi khả năng tính toán song song cao trong các lớp tích chập. Việc tăng tốc bằng FPGA thông qua công nghệ HLS giúp rút ngắn chu kỳ phát triển mà vẫn đảm bảo hiệu năng phần cứng tốt.

1.2. Mục tiêu đề tài

- Xây dựng mô hình Convolutional Autoencoder khử nhiễu ảnh MNIST bằng Python/Keras.
- Chuyển đổi mô hình sang C++ với kiểu dữ liệu Fixed-point phù hợp với HLS.
- Sử dụng Vitis HLS để tổng hợp IP Core với các kỹ thuật tối ưu hóa Pipeline, Loop Unroll, Inline.
- Tích hợp IP vào hệ thống SoC trên Vivado với giao tiếp AXI-Lite và AXI-MM (BRAM).
- Viết phần mềm điều khiển C chạy trên ARM KV260, thực nghiệm với 100 ảnh kiểm thử.
- Đánh giá kết quả thông qua các chỉ số PSNR, MSE và so sánh với output Python reference.

Chương 2. CƠ SỞ LÝ THUYẾT

2.1. Mạng Autoencoder Tích chập (CNN Autoencoder)

2.1.1. Khái niệm Autoencoder

Autoencoder là một kiến trúc mạng nơ-ron học không giám sát, gồm hai phần chính: Encoder (mã hóa) và Decoder (giải mã). Encoder nén đầu vào thành một biểu diễn ẩn chiều thấp hơn (latent representation), trong khi Decoder tái tạo lại đầu vào từ biểu diễn đó.

$$x \rightarrow \text{Encoder}(x) = z \rightarrow \text{Decoder}(z) = \hat{x}$$

Trong đó

- x : ảnh đầu vào
- z : đặc trưng ẩn (latent feature)
- $\hat{x}$: ảnh tái tạo

2.1.2. Mạng khử nhiễu cổ điển (Denoising Autoencoder - DAE)

Denoising Autoencoder (DAE) được Vincent và cộng sự đề xuất tại hội nghị ICML năm 2008 nhằm giải quyết bài toán học biểu diễn dữ liệu có khả năng chống nhiễu. Khác với Autoencoder truyền thống chỉ học tái tạo lại chính dữ liệu đầu vào, DAE được huấn luyện để khôi phục ảnh sạch từ ảnh đã bị thêm nhiễu. Giả sử ảnh gốc là x , quá trình thêm nhiễu được mô tả bởi:

$$\tilde{x} \sim q_D(\tilde{x}|x)$$

Trong đó:

- x : ảnh gốc.
- $\tilde{x}$: ảnh sau khi thêm nhiễu.
- q_D : quá trình sinh nhiễu.

Nguyên lý bản thiết kế Denoising Autoencoder (DAE) cổ điển được định nghĩa dựa trên các lớp kết nối đầy đủ (Fully-Connected / Dense Layer). Luồng toán học tuân theo phương trình:

Encoder

$$y = s(W\tilde{x} + b)$$

Decoder

$$z = s(W'y + b')$$

Loss

$$L(x, z) = ||x - z||^2$$

Tuy nhiên, khi áp dụng cho xử lý ảnh và đặc biệt là triển khai trên FPGA, kiến trúc Dense bộc lộ nhiều hạn chế như số lượng tham số lớn, không khai thác được cấu trúc không gian của ảnh và tiêu tốn nhiều tài nguyên phần cứng.

Bảng 1 So sánh Dense DAE và Convolutional Autoencoder

Tiêu chí	Dense DAE	CNN Autoencoder
Khai thác cấu trúc ảnh	Không	Có
Weight sharing	Không	Có
Số tham số (MNIST 28×28)	400k – 1M	665
Khả năng triển khai FPGA	Khó	Phù hợp
Hiệu quả với ảnh MNIST	Trung bình	Tốt hơn

2.1.3. Convolutional Autoencoder (CAE)

Để khắc phục toàn bộ các hạn chế cốt lõi của kiến trúc Dense DAE cổ điển và đảm bảo tính khả thi khi triển khai trên phần cứng SoC, đề án này đề xuất cấu trúc Mạng tự mã hóa tích chập (Convolutional Autoencoder - CAE). Thay vì sử dụng các lớp kết nối đầy đủ (Dense layer), mạng CAE tái cấu trúc hệ thống dựa trên sự kết hợp của các lớp Tích chập 2D (Conv2D), Trích xuất cực đại (MaxPooling2D), và Tăng mẫu không gian (UpSampling2D).

Kiến trúc mạng nơ-ron tích chập (CNN) mang lại những đặc tính toán học ưu việt, đồng thời mở ra các giải pháp tối ưu hóa tài nguyên phần cứng một cách triệt để

2.1.4. Phép toán tích chập (Convolution)

Phép tích chập 2D là thành phần cốt lõi của mạng CNN Autoencoder. Tại mỗi vị trí, kernel trượt qua feature map và thực hiện phép nhân-tích lũy (MAC):

$$G[x, y, n] = \sum_i \sum_j \sum_k input[x+i-1][y+j-1][k] \times kernel[n][k][i][j] + bias[n]$$

Trong đó:

- Input là feature map đầu vào.
- Kernel là bộ lọc học được.
- Bias là hệ số dịch.
- Output là feature map sau tích chập.

Trong đề án, kiến trúc sử dụng 4 lớp tích chập với kernel 3×3, padding SAME để giữ nguyên kích thước không gian.

2.1.5. Hàm kích hoạt

Hàm ReLU (Rectified Linear Unit) được sử dụng sau các lớp Conv2D trung gian, giúp mạng học các đặc trưng phi tuyến:

$$ReLU(x) = \max(0, x)$$

Hàm Sigmoid được sử dụng ở lớp đầu ra để nén giá trị pixel về khoảng [0,1], phù hợp với ảnh đã chuẩn hóa:

$$Sigmoid(x) = 1 / (1 + e^{(-x)})$$

2.1.6. MaxPooling

MaxPooling là phép giảm kích thước feature map bằng cách lấy giá trị lớn nhất trong từng vùng cửa sổ. Trong đồ án sử dụng cửa sổ 2 × 2 với stride bằng 2, giúp giảm kích thước dữ liệu và số phép tính, đồng thời làm tăng khả năng chống nhiễu

2.1.7. UpSampling

UpSampling làm tăng kích thước feature map sau khi Encoder đã nén dữ liệu. Trong đồ án sử dụng phương pháp Nearest Neighbor với hệ số 2. Đóng vai trò để phục hồi kích thước ảnh cũng như chuẩn bị cho Decoder.

2.1.8. Kiến trúc Encoder – Decoder trong đồ án

Mô hình Denoising Autoencoder trong đồ án bao gồm:

Bảng 2 Kiến trúc mạng Denoising Autoencoder

Tầng	Kích thước đầu vào	Kích thước đầu ra	Tham số
Input	28×28×1	28×28×1	0
Conv2D_1 (4 filter, 3×3)	28×28×1	28×28×4	40
MaxPool 2×2	28×28×4	14×14×4	0
Conv2D_2 (8 filter, 3×3)	14×14×4	14×14×8	296
UpSample 2×2	14×14×8	28×28×8	0
Conv2D_3 (4 filter, 3×3)	28×28×8	28×28×4	292
Conv2D_out (1 filter, 3×3, Sigmoid)	28×28×4	28×28×1	37
Tổng tham số			665

2.2. Tổng quan về công nghệ FPGA KV260

2.2.1. Kiến trúc Zynq UltraScale+ MPSoC

Xilinx KV260 được xây dựng trên chip Zynq UltraScale+ MPSoC (xck26), tích hợp hai phần trên cùng một die silicon:

- Processing System (PS): Bộ vi xử lý ARM Cortex-A53 quad-core 64-bit, chạy hệ điều hành Linux, quản lý giao tiếp I/O và điều khiển PL.
- Programmable Logic (PL): Ma trận FPGA Kintex UltraScale+ với các DSP Slice, BRAM, LUT và Flip-flop có thể lập trình để tạo hardware accelerator.

Hai phần PS và PL giao tiếp thông qua chuẩn AXI4, cho phép CPU điều khiển IP Core và truyền dữ liệu với băng thông cao.

2.2.2. Ưu điểm FPGA trong triển khai neural network

- Tính song song cao: Hàng trăm phép nhân-cộng thực hiện đồng thời bằng DSP Slice.
- Độ trễ thấp và có thể dự đoán: Phù hợp cho các ứng dụng thời gian thực.
- Tiêu thụ điện năng thấp hơn GPU trong khi duy trì thông lượng cao.
- Tùy biến kiến trúc: Điều chỉnh độ rộng bit, pipeline depth theo nhu cầu ứng dụng.

2.3. High-Level Synthesis (HLS)

2.3.1. Khái niệm HLS

High-Level Synthesis (HLS) là phương pháp thiết kế phần cứng cho phép chuyển đổi mô tả thuật toán ở mức cao (thường sử dụng ngôn ngữ C, C++ hoặc SystemC) thành mã mô tả phần cứng RTL như Verilog hoặc VHDL một cách tự động.

So với phương pháp thiết kế RTL truyền thống, HLS giúp rút ngắn đáng kể thời gian phát triển, giảm độ phức tạp của quá trình thiết kế và tạo điều kiện thuận lợi cho việc kiểm thử, sửa lỗi cũng như tối ưu hệ thống. Bên cạnh đó, việc thay đổi thuật toán hoặc kiến trúc mạng chỉ cần thực hiện trên mã nguồn C/C++ mà không phải chỉnh sửa toàn bộ mã RTL.

2.3.2. Kiểu dữ liệu Fixed-point `ap_fixed`

Trong các ứng dụng học sâu, dữ liệu và trọng số thường được biểu diễn dưới dạng số thực dấu phẩy động (Floating-point) nhằm đạt độ chính xác cao. Tuy nhiên, việc xử lý Floating-point trên FPGA yêu cầu nhiều tài nguyên logic, tiêu thụ nhiều năng lượng và làm tăng độ trễ tính toán.

Để khắc phục hạn chế này, HLS cung cấp thư viện số cố định `ap_fixed`, cho phép biểu diễn các số thực bằng một số lượng bit xác định. Dạng tổng quát của kiểu dữ liệu này là:

ap_fixed < W, I >

Trong đó:

- W (Word Length): tổng số bit của dữ liệu.
- I (Integer Length): số bit dành cho phần nguyên (bao gồm cả bit dấu).
- Phần còn lại được sử dụng để biểu diễn phần thập phân.

Việc sử dụng số cố định trong thiết kế phần cứng giúp giảm tài nguyên sử dụng (LUT, DSP và BRAM), cũng như lưu lượng truyền dữ liệu và dung lượng bộ nhớ, tăng tốc độ xử lý nhờ các phép toán đơn giản hơn so với Floating-point.

2.3.3. Các pragma tối ưu hóa HLS

Ngoài khả năng chuyển đổi từ C/C++ sang RTL, HLS còn hỗ trợ nhiều chỉ thị (Pragmas) giúp người thiết kế điều khiển cách phần cứng được sinh ra. Các pragma không làm thay đổi chức năng của thuật toán nhưng ảnh hưởng trực tiếp đến kiến trúc phần cứng, mức sử dụng tài nguyên và hiệu năng của hệ thống.

- **Pipeline:** cho phép chồng lấp quá trình thực hiện các phép tính, tăng thông lượng của hệ thống.
- **Loop Unroll:** mở rộng các vòng lặp để thực hiện nhiều phép tính song song.
- **Inline:** tích hợp các hàm con vào hàm chính nhằm giảm chi phí gọi hàm và tối ưu logic.
- **Array Partition:** chia nhỏ mảng thành nhiều khối nhớ độc lập để tăng số lượng truy cập đồng thời.
- **Dataflow:** cho phép nhiều khối chức năng hoạt động song song theo mô hình đường ống.
- **Interface Pragmas:** xác định kiểu giao tiếp giữa IP Core với các thành phần khác của hệ thống như AXI4-Lite, AXI4 Master hoặc AXI4-Stream.

Việc lựa chọn và kết hợp các pragma phù hợp giúp cân bằng giữa tốc độ xử lý, mức sử dụng tài nguyên và tần số hoạt động của IP Core.

2.3.4. Chuẩn giao tiếp AXI

Để bộ tăng tốc mạng Autoencoder hoạt động đồng bộ và khép kín trên một hệ thống SoC (System-on-Chip) hoàn chỉnh, nó cần thiết lập các kênh truyền thông tiêu chuẩn với bộ vi xử lý ARM lõi cứng và các phân vùng bộ nhớ ngoại. Chuẩn giao tiếp AMBA AXI được áp dụng để thiết lập các liên kết vật lý này:

- **AXI4-Lite:** sử dụng cho việc truyền dữ liệu dung lượng nhỏ như các thanh ghi điều khiển, cấu hình và trạng thái của IP Core.
- **AXI4 (Memory-Mapped):** hỗ trợ cơ chế đọc/ghi dữ liệu theo cụm (Burst Read/Burst Write) trực tiếp từ bộ nhớ hệ thống vào các vùng đệm Dual-Port BRAM nội tại với IP Core mà không cần thông qua sự can thiệp liên tục từ CPU, giảm tải tối đa cho hệ điều hành nhúng

Nhờ chuẩn AXI, các IP được sinh bởi HLS có thể dễ dàng tích hợp vào Vivado Block Design và giao tiếp với bộ xử lý ARM hoặc các bộ nhớ trong hệ thống SoC.

2.4. Tập dữ liệu MNIST và bài toán khử nhiễu

2.4.1. Tập dữ liệu MNIST

MNIST (Modified National Institute of Standards and Technology) là bộ dữ liệu chuẩn được sử dụng rộng rãi trong các nghiên cứu về học sâu và thị giác máy tính. Bộ dữ liệu gồm 70.000 ảnh chữ số viết tay từ 0 đến 9, trong đó có 60.000 ảnh huấn luyện và 10.000 ảnh kiểm thử. Mỗi ảnh là ảnh xám (grayscale) có kích thước 28×28 pixel, với giá trị mỗi pixel nằm trong khoảng từ 0 đến 255.

2.4.2. Bài toán khử nhiễu ảnh

Khử nhiễu ảnh (Image Denoising) là quá trình loại bỏ các thành phần nhiễu khỏi ảnh để khôi phục lại ảnh gốc. Trong đề án này, bài toán được giải quyết bằng Denoising Autoencoder, trong đó ảnh nhiễu được sử dụng làm đầu vào và ảnh gốc sạch được sử dụng làm đầu ra mong muốn

Mạng nơ-ron sẽ học mối quan hệ giữa ảnh nhiễu và ảnh sạch để tự động loại bỏ các thành phần nhiễu trong quá trình suy luận.

2.4.3. Quy trình tiền xử lý dữ liệu

Trước khi đưa vào mô hình, dữ liệu MNIST cần trải qua một số bước tiền xử lý nhằm đảm bảo quá trình huấn luyện và suy luận đạt hiệu quả cao:

- Chuẩn hóa giá trị pixel từ khoảng $[0, 255]$ về $[0, 1]$.
- Chuyển ảnh về kích thước $28 \times 28 \times 1$ để phù hợp với đầu vào của mạng CNN.
- Sinh ảnh nhiễu bằng cách bổ sung Gaussian Noise lên ảnh gốc.
- Giới hạn giá trị pixel về khoảng $[0, 1]$ sau khi thêm nhiễu để đảm bảo dữ liệu hợp lệ.

Sau quá trình tiền xử lý, cặp dữ liệu (ảnh nhiễu, ảnh gốc) được sử dụng để huấn luyện mạng Autoencoder học cách tái tạo ảnh sạch từ ảnh bị nhiễu.

Chương 3. TRIỂN KHAI ĐỀ TÀI

3.1. Thiết kế và lựa chọn kiến trúc mô hình Python

3.1.1. Môi trường và cấu hình huấn luyện

Mô hình Denoising Autoencoder được xây dựng bằng Python với thư viện Keras. Sau khi hoàn thành quá trình huấn luyện, toàn bộ trọng số sẽ được trích xuất để phục vụ cho bước chuyển đổi sang C++ và triển khai trên FPGA bằng Vitis HLS.

Quá trình huấn luyện của cả hai mô hình đều sử dụng cùng một cấu hình nhằm đảm bảo tính khách quan trong quá trình so sánh.

- Dataset: MNIST (60.000 train / 10.000 test), ảnh xám 28×28.
- Nhiễu Gaussian: $\tilde{x} = x + n$, $n \sim N(0, \sigma^2)$, $\sigma = 0.5$.
- Batch size: 128, Epoch: 50, Optimizer: Adam, Learning Rate: 0.001, Loss: Mean Squared Error (MSE).

3.1.2. Hai phương án kiến trúc được đề xuất

Trong quá trình thiết kế, hai kiến trúc với độ phức tạp khác nhau được xây dựng và đánh giá song song nhằm tìm hiểu mối quan hệ giữa độ phức tạp mô hình và hiệu năng khử nhiễu, từ đó lựa chọn kiến trúc phù hợp nhất cho triển khai FPGA.

a. Kiến trúc mô hình nhỏ (Lightweight)

Kiến trúc thứ nhất được thiết kế theo hướng đơn giản nhằm giảm số lượng tham số và tài nguyên phần cứng. Mạng gồm hai lớp tích chập ở Encoder và hai lớp tích chập ở Decoder, kết hợp với một lớp MaxPooling và một lớp UpSampling, gồm các lớp:

Bảng 3 Kiến trúc mô hình nhỏ (665 tham số)

Tầng	Kích thước ra	Tham số
Input	28×28×1	0
Conv2D_1 (4 filter, 3×3, ReLU)	28×28×4	40
MaxPool 2×2	14×14×4	0
Conv2D_2 (8 filter, 3×3, ReLU)	14×14×8	296
UpSample 2×2	28×28×8	0
Conv2D_3 (4 filter, 3×3, ReLU)	28×28×4	292
Conv2D_out (1 filter, 3×3, Sigmoid)	28×28×1	37

Tổng		665
-------------	--	------------

b. Kiến trúc mô hình lớn (Optimized)

Nhằm cải thiện khả năng khử nhiễu, một kiến trúc sâu hơn cũng được xây dựng. So với mô hình nhỏ, kiến trúc này tăng số lượng bộ lọc ở mỗi tầng, bổ sung thêm các lớp Batch Normalization và mở rộng phần Bottleneck.

Batch Normalization giúp chuẩn hóa phân bố đặc trưng trong quá trình huấn luyện, từ đó tăng tốc độ hội tụ và cải thiện khả năng tổng quát của mô hình. Tuy nhiên, việc bổ sung các lớp này cũng làm tăng số lượng tham số và độ phức tạp khi triển khai trên FPGA.

Bảng 4 Kiến trúc mô hình lớn (~12.000 tham số)

Tầng	Kích thước đầu ra	Tham số
Input	28×28×1	0
Conv2D_1 (8 filter, 3×3)	28×28×8	80
Batch Normalization	28×28×8	32
MaxPooling 2×2	14×14×8	0
Conv2D_2 (16 filter, 3×3)	14×14×16	1,168
Batch Normalization	14×14×16	64
MaxPooling 2×2	7×7×16	0
Conv2D_3 (32 filter, 3×3)	7×7×32	4,640
Batch Normalization	7×7×32	128
UpSampling 2×2	14×14×32	0
Conv2D_4 (16 filter, 3×3)	14×14×16	4,624
Batch Normalization	14×14×16	64
UpSampling 2×2	28×28×16	0
Conv2D_5 (8 filter, 3×3)	28×28×8	1,160
Batch Normalization	28×28×8	32
Conv2D_out (1 filter, 3×3, Sigmoid)	28×28×1	73
Tổng		≈ 12.065

3.1.3. So sánh kết quả huấn luyện hai mô hình

Sau 50 epoch huấn luyện trên cùng tập dữ liệu, hai mô hình được đánh giá trên 10.000 ảnh test với cùng cấu hình nhiễu:

Bảng 5 So sánh kết quả huấn luyện Model nhỏ và Model lớn

Chỉ số	Model nhỏ (665 tham số)	Model lớn (~12.000 tham số)
PSNR ảnh nhiễu (đầu vào)	9.37 dB	9.37 dB
PSNR ảnh sau khử nhiễu	18.83 dB	19.76 dB
Cải thiện PSNR	+9.46 dB	+10.39 dB
MSE ảnh nhiễu	0.115587	0.115563
MSE ảnh sau khử nhiễu	0.013091	0.010719
MSE giảm	88.29%	90.72%

3.1.4. Phân tích và quyết định lựa chọn kiến trúc cho FPGA

Bảng 6 Phân tích đánh đổi và quyết định lựa chọn kiến trúc

	Model lớn	Model nhỏ (→ FPGA)
Tổng tham số	~12.000	665 (ít hơn 18 lần)
PSNR khử nhiễu	19.76 dB	18.83 dB
Chênh lệch PSNR so với model lớn	—	−0.93 dB
Batch Normalization	Có	Không
Độ phức tạp tính toán	Cao	Thấp
Bộ nhớ yêu cầu	Nhiều	Ít
DSP sử dụng	Nhiều	Ít
Khả năng tổng hợp HLS	Khó	Tốt

Kết quả thực nghiệm cho thấy model lớn chỉ cải thiện PSNR thêm 0.93 dB (tương đương cải thiện MSE thêm 2.43%) so với model nhỏ, trong khi số lượng tham số tăng gấp 18 lần (665 → 12.000). Mức cải thiện chất lượng khử nhiễu không tương xứng với chi phí tài nguyên phần cứng phải trả - đặc biệt với BRAM và DSP vốn là tài nguyên hạn chế trên FPGA cỡ nhỏ như KV260 (144 khối BRAM, 1248 DSP). Do đó, đồ án quyết định lựa chọn model nhỏ làm kiến trúc chính thức cho toàn bộ quy trình chuyển đổi sang HLS và triển khai phần cứng.

3.2. Chuyển đổi mô hình sang C++ và tối ưu HLS

3.2.1. Quy trình chuyển đổi trọng số Python sang C++

Sau khi huấn luyện mô hình Autoencoder, các trọng số float32 của model nhỏ được chuyển đổi sang `ap_fixed<16,6>` và dùng Python với thư viện `h5py` để trích xuất kernel weights, biases cho từng layer convolution (conv1, conv2, conv3, conv_out). Các giá trị được xuất ra thành các file header (.h) riêng biệt. Việc xuất riêng từng layer giúp thuận tiện cho quá trình ánh xạ phần cứng trong HLS.

. Định dạng lưu trữ kernel theo thứ tự `[output_channel][input_channel][kH][kW]` giúp HLS đọc tuần tự từ bộ nhớ, hỗ trợ tổng hợp pipeline hiệu quả hơn.

Bảng 7 Các file header chứa trọng số Fixed-point

File Header	Số kernel	Số bias
conv1_weights.h	36 (4×1×3×3)	4
conv2_weights.h	288 (8×4×3×3)	8
conv3_weights.h	288 (4×8×3×3)	4
conv_out_weights.h	36 (1×4×3×3)	1

3.2.2. Cấu trúc mã nguồn C++ HLS

Sau khi chuyển đổi, chương trình được chia thành nhiều tệp nhằm tách biệt giữa mô tả thuật toán, trọng số và chương trình kiểm thử.

 conv_out_weights.h	5/9/2026 1:02 PM	C/C++ Header	1 KB
 conv1_weights.h	5/9/2026 12:56 PM	C/C++ Header	2 KB
 conv2_weights.h	5/9/2026 1:20 PM	C/C++ Header	7 KB
 conv3_weights.h	5/9/2026 7:22 PM	C/C++ Header	7 KB
 denoiser	5/9/2026 8:00 PM	CPP File	9 KB
 denoiser.h	5/9/2026 7:52 PM	C/C++ Header	3 KB
 main	5/9/2026 8:00 PM	CPP File	5 KB
 top	5/9/2026 7:53 PM	CPP File	2 KB

Hình 1 Cấu trúc mã nguồn C++ HLS

Mã nguồn C++ được tổ chức thành các file sau:

- denoiser.h: Định nghĩa kiểu `fixed_t = ap_fixed<16,6>`, class `Denoiser` với các hàm `conv2d_1`, `maxpool_2x2_4ch`, `conv2d_2`, `upsample_2x2_8ch`, `conv2d_3`, `conv2d_out`, `inference`. Khai báo hàm `sigmoid` sử dụng `hls::exp` nội tại.
- denoiser.cpp: Hiện thực các hàm tính toán với `pragma HLS INLINE`, `UNROLL`. Sử dụng buffer trung gian `buffer_A[28][28][8]` và `buffer_B[28][28][8]` để tái sử dụng bộ nhớ.

- top.cpp: Hàm top-level denoiser_top(fixed_t input[784], fixed_t output[784]) với interface m_axi + s_axilite.
- main.cpp (testbench): Đọc 100 ảnh từ file .txt, gọi denoiser_top, tính PSNR so sánh với Python gold output.

3.2.3. Tối ưu hóa mô hình cho HLS

Để mô hình phù hợp với việc triển khai trên FPGA, một số kỹ thuật tối ưu được áp dụng trong Vitis HLS:

- **Chuyển đổi sang số fixed-point:** Trọng số và dữ liệu trung gian được chuyển từ floating-point sang fixed-point nhằm giảm tài nguyên tính toán và bộ nhớ, đồng thời vẫn duy trì độ chính xác của mô hình.
- **Tái sử dụng bộ nhớ (Shared Buffer):** Hai bộ đệm trung gian được sử dụng luân phiên giữa các lớp tích chập thay vì cấp phát riêng cho từng lớp, giúp giảm đáng kể tài nguyên BRAM.
- **Áp dụng pragma HLS:** Các chỉ thị **INLINE**, **PIPELINE** và **UNROLL** được sử dụng để tối ưu quá trình tổng hợp phần cứng, tăng khả năng xử lý song song và cải thiện thông lượng.
- **Tối ưu truy cập bộ nhớ:** Thứ tự các vòng lặp được sắp xếp lại để việc đọc trọng số và dữ liệu diễn ra tuần tự hơn, giúp HLS tối ưu quá trình truy cập bộ nhớ.

Các kỹ thuật trên giúp cân bằng giữa hiệu năng xử lý và lượng tài nguyên FPGA sử dụng, đáp ứng yêu cầu triển khai mô hình trên nền tảng KV260.

3.2.4. So sánh các cấu hình pragma tối ưu hóa

Đồ án thử nghiệm hệ thống ba cấu hình pragma khác nhau để tìm điểm cân bằng giữa latency và tài nguyên phần cứng:

1. Baseline (không pragma ngoài interface): HLS tự do lựa chọn kiến trúc, dùng để làm mốc tham chiếu.
2. Tối ưu tốc độ (Performance-Oriented): PIPELINE II=1 toàn bộ + UNROLL hoàn toàn + ARRAY_PARTITION complete + kiến trúc 5 buffer độc lập (buf1-buf5) loại bỏ phụ thuộc dữ liệu tuần tự.
3. Tối ưu tài nguyên (Area-Efficient, được chọn): INLINE recursive + UNROLL factor=2 + Shared Buffer (buffer_A, buffer_B).

Bảng 8 So sánh ba cấu hình pragma tối ưu hóa HLS

Tài nguyên	Baseline (không pragma)	Tối ưu tốc độ (Full Pipeline+Unroll)	Tối ưu tài nguyên (INLINE+UNROLL×2) - Chọn
BRAM	23	105 (+356%)	24
DSP	94	157 (+67%)	16 (-83%)
FF	7.209	16.834 (+133%)	3.475 (-51%)
LUT	15.584	26.749 (+72%)	10.059 (-35%)
Fmax	136.99 MHz	136.99 MHz	136.99 MHz
Latency	~428.000 cycles	~26.000 cycles	~540.000 cycles
Thời gian/ảnh	—	~0.197 ms	~3.95 ms

Modules & Loops	Issue Type	Latency(cycles)	Latency(ns)	Latency	Interval	rip Count	Pipelined	BRAM	DSP	FF	LUT	URAM
denoiser_top		540210	5.402E6	-	540211	-	no	24	16	3475	10059	0
denoiser_top_Pipeline_VITIS_LOOP_18_1		787	7.870E3	-	787	-	no	0	0	51	77	0
inference		538618	5.386E6	-	538618	-	no	6	16	1599	6540	0
inference_Pipeline_VITIS_LOOP_45_1_VITIS_1	II Violation	2354	2.354E4	-	2354	-	no	0	0	87	711	0
inference_Pipeline_VITIS_LOOP_97_1_VITIS_1	II Violation	7842	7.842E4	-	7842	-	no	0	0	114	644	0
loop_oc_1_loop_h_1_loop_w_1	II Violation	137984	1.380E6	44	-	3136	no	-	-	-	-	-
loop_oc_2_loop_h_2_loop_w_2	II Violation	70560	7.060E5	45	-	1568	no	-	-	-	-	-
loop_oc_3_loop_h_3_loop_w_3	II Violation	254016	2.540E6	81	-	3136	no	-	-	-	-	-
loop_h_out_loop_w_out		65856	6.590E5	84	-	784	no	-	-	-	-	-
inference_Pipeline_loop_kh_out_loop_kw	II Violation	41	410.000	-	41	-	no	0	2	75	745	0
denoiser_top_Pipeline_VITIS_LOOP_25_2		787	7.870E3	-	787	-	no	0	0	31	77	0

Hình 2 Cấu hình tối ưu tài nguyên

Cấu hình tối ưu tốc độ đạt latency thấp hơn khoảng 20 lần so với cấu hình tối ưu tài nguyên, nhưng đổi lại BRAM tăng gần 5 lần (24→105) và DSP tăng gần 10 lần (16→157). Việc sử dụng 5 buffer độc lập thay vì tái sử dụng 2 buffer là nguyên nhân chính khiến BRAM tăng mạnh, do mỗi buffer có thể áp dụng ARRAY_PARTITION complete độc lập, loại bỏ hoàn toàn xung đột cổng truy cập và đạt II=1 cho mọi vòng lặp.

Pragma HLS INLINE recursive trong cấu hình được chọn giúp phẳng hóa toàn bộ logic vào một khối duy nhất, cho phép HLS nhận ra các lớp chạy tuần tự có thể tái sử dụng chung các khối DSP, từ đó giảm DSP đáng kể (94 → 16). Pragma UNROLL factor=2 chỉ tạo 2 bộ nhân song song thay vì 1 (không unroll) hoặc 8 (full unroll), giúp giảm mạnh FF và LUT nhưng làm tăng latency.

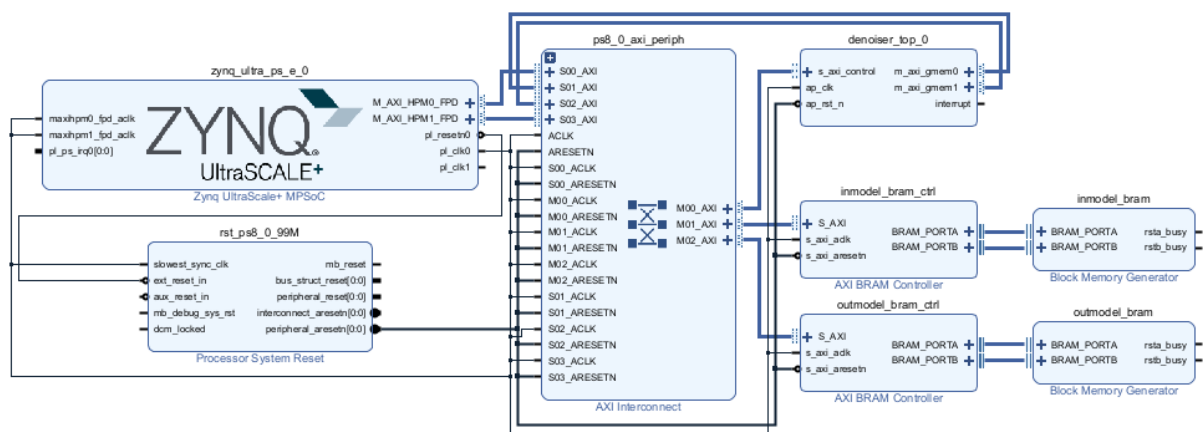
Với mạng kích thước nhỏ như MNIST Autoencoder (665 tham số), việc tối ưu tốc độ tối đa không mang lại lợi ích tương xứng với chi phí tài nguyên phải trả. Đồ án lựa chọn cấu hình tối ưu tài nguyên (INLINE recursive + UNROLL factor=2 + Shared Buffer) làm kiến trúc cuối cùng, đạt latency ~3.95 ms/ảnh - vẫn đáp ứng tốt yêu cầu xử

lý ảnh MNIST trong khi tiết kiệm đáng kể tài nguyên FPGA, phù hợp triển khai trên thiết bị nhúng giá rẻ

3.3. Tích hợp IP vào hệ thống SoC

3.3.1. Kiến trúc hệ thống SoC

Sau khi hoàn thành Vitis HLS, mô hình Autoencoder được xuất sang Vivado dưới dạng IP để tích hợp vào hệ thống SoC trên nền tảng Zynq UltraScale+ MPSoC. Hệ thống SoC được xây dựng trong Vivado 2022.2 với các thành phần chính như sau:



Hình 3 Hệ thống SoC hoàn chỉnh

1. Zynq UltraScale+ MPSoC (PS): Khối xử lý trung tâm, chạy Linux Ubuntu trên ARM Cortex-A53, chịu trách nhiệm điều khiển IP Core, quản lý bộ nhớ và giao tiếp với phần cứng thông qua bus AXI.
2. denoiser_top_0 (IP Core HLS): IP sinh từ Vitis HLS. Có 2 cổng m_axi (gmem0 cho input, gmem1 cho output) và 1 cổng s_axi_control cho điều khiển AP, sử dụng hai nhóm giao tiếp chính:
 - **AXI4-Lite (s_axi_control):** dùng để CPU ghi các thanh ghi điều khiển như ap_start, đọc trạng thái ap_done, ap_idle và truyền địa chỉ của vùng nhớ đầu vào, đầu ra.
 - **AXI4 Master (m_axi):** dùng để IP chủ động đọc dữ liệu ảnh từ BRAM đầu vào và ghi kết quả sang BRAM đầu ra mà không cần CPU truyền từng điểm ảnh.

Nhờ cơ chế Master AXI, toàn bộ dữ liệu được truyền theo dạng burst nên giảm đáng kể thời gian truy cập bộ nhớ.

3. AXI BRAM Controller (inmodel_bram_ctrl): Cầu nối AXI ↔ BRAM cho dữ liệu ảnh đầu vào.
4. AXI BRAM Controller (weights_bram_ctrl): Cầu nối AXI ↔ BRAM cho dữ liệu đầu ra.
5. Block Memory Generator: Lưu trữ dữ liệu trung gian phục vụ quá trình suy luận.
6. AXI Interconnect: Thực hiện kết nối giữa bộ xử lý PS, IP Core và các bộ điều khiển BRAM.

3.3.2. Address Mapping

Sau khi hoàn thành kết nối phần cứng, Vivado tự động gán địa chỉ cho các ngoại vi thông qua Address Editor. Các vùng địa chỉ chính của hệ thống được sử dụng trong đồ án được trình bày như sau:

Name	Interface	Slave Segment	Master Base Address	Range	Master High Address
/denoiser_top_0/Data_m_axi_gmem0 (64 address bits : 16E)					
/inmodel_bram_ctrl/S_AXI	S_AXI	Mem0	0x0000_0000_A001_0000	8K	0x0000_0000_A001_1FFF
/outmodel_bram_ctrl/S_AXI	S_AXI	Mem0	0x0000_0000_A001_2000	8K	0x0000_0000_A001_3FFF
Excluded (1)					
/denoiser_top_0/s_axi_control	s_axi_control	Reg	0x0000_0000_A000_0000	64K	0x0000_0000_A000_FFFF
/denoiser_top_0/Data_m_axi_gmem1 (64 address bits : 16E)					
/inmodel_bram_ctrl/S_AXI	S_AXI	Mem0	0x0000_0000_A001_0000	8K	0x0000_0000_A001_1FFF
/outmodel_bram_ctrl/S_AXI	S_AXI	Mem0	0x0000_0000_A001_2000	8K	0x0000_0000_A001_3FFF
Excluded (1)					
/denoiser_top_0/s_axi_control	s_axi_control	Reg	0x0000_0000_A000_0000	64K	0x0000_0000_A000_FFFF
/zynq_ultra_ps_e_0					
/zynq_ultra_ps_e_0/Data (40 address bits : 0x00A0000000 [256M], 0x0400000000 [4G], 0x1000000000 [224G], 0x00B0000000 [256M], 0x0500000000 [4G], 0x48000000)					
/denoiser_top_0/s_axi_control	s_axi_control	Reg	0x00_A000_0000	64K	0x00_A000_FFFF
/denoiser_top_0/s_axi_control	s_axi_control	Reg	0x00_A000_0000	64K	0x00_A000_FFFF
/inmodel_bram_ctrl/S_AXI	S_AXI	Mem0	0x00_A001_0000	8K	0x00_A001_1FFF
/inmodel_bram_ctrl/S_AXI	S_AXI	Mem0	0x00_A001_0000	8K	0x00_A001_1FFF
/outmodel_bram_ctrl/S_AXI	S_AXI	Mem0	0x00_A001_2000	8K	0x00_A001_3FFF
/outmodel_bram_ctrl/S_AXI	S_AXI	Mem0	0x00_A001_2000	8K	0x00_A001_3FFF

Hình 4 Địa chỉ AXI của hệ thống SoC

1. Vùng điều khiển (Control Space)

Đây là vùng địa chỉ mà CPU sử dụng để điều khiển hoạt động của IP Core. **denoiser_top_0/s_axi_control** có địa chỉ cơ sở 0xA0000000, kích thước 64 KB. Vùng này chứa các thanh ghi điều khiển như ap_start, ap_done, ap_idle, địa chỉ BRAM đầu vào, địa chỉ BRAM đầu ra. CPU ghi các giá trị vào các thanh ghi này trước khi kích hoạt IP.

2. Vùng dữ liệu (Data Space)

Đây là vùng bộ nhớ dùng để trao đổi dữ liệu giữa CPU và IP.

- **Input BRAM** có địa chỉ cơ sở 0xA0010000, kích thước 8 KB. Lưu ảnh nhiễu đầu vào.
- **Output BRAM** có địa chỉ cơ sở 0xA0012000, kích thước 8 KB. Lưu ảnh sau khi khử nhiễu.

CPU ghi dữ liệu vào Input BRAM trước khi chạy IP và đọc kết quả từ Output BRAM sau khi quá trình suy luận hoàn tất.

3. Cơ chế truy cập bộ nhớ

IP Core sử dụng **hai cổng AXI Master độc lập**:

- **gmem0** được sử dụng để đọc dữ liệu từ Input BRAM.
- **gmem1** được sử dụng để ghi dữ liệu sang Output BRAM.

Việc sử dụng hai cổng AXI riêng biệt giúp quá trình đọc và ghi dữ liệu có thể diễn ra đồng thời, giảm hiện tượng tranh chấp bộ nhớ và cải thiện hiệu năng của hệ thống.

3.3.3. Thanh ghi AXI-Lite điều khiển

IP Core được điều khiển thông qua giao tiếp **AXI4-Lite**, bao gồm các thanh ghi điều khiển và thanh ghi chứa địa chỉ vùng nhớ đầu vào, đầu ra.

Bảng 9 Bảng thanh ghi AXI-Lite từ denoiser_top_control_s_axi.v

Offset	Tên thanh ghi	Chức năng
0x00	AP_CTRL	Bit0=ap_start, Bit1=ap_done(COR), Bit2=ap_idle
0x10	INPUT_ADDR_LO	Địa chỉ thấp 32-bit của con trỏ input (BRAM_IN)
0x14	INPUT_ADDR_HI	Địa chỉ cao 32-bit (= 0)
0x1C	OUTPUT_ADDR_LO	Địa chỉ thấp 32-bit của con trỏ output (BRAM_OUT)
0x20	OUTPUT_ADDR_HI	Địa chỉ cao 32-bit (= 0)

Quá trình điều khiển IP được thực hiện bằng cách ghi địa chỉ của vùng nhớ đầu vào và đầu ra vào các thanh ghi tương ứng, sau đó kích hoạt tín hiệu ap_start. Khi IP hoàn thành xử lý, tín hiệu ap_done được thiết lập để thông báo cho bộ xử lý đọc kết quả từ BRAM.

3.4. Phần mềm điều khiển trên ARM (KV260)

3.4.1. Phương pháp truy cập phần cứng

Sau khi hoàn thành thiết kế phần cứng và tích hợp IP Core vào hệ thống SoC, một chương trình điều khiển được xây dựng để thực hiện quá trình trao đổi dữ liệu giữa

bộ xử lý ARM và IP Core. Chương trình được viết bằng ngôn ngữ C và chạy trên hệ điều hành Linux Ubuntu của kit KV260.

Để truy cập trực tiếp các thanh ghi điều khiển và vùng nhớ BRAM từ không gian người dùng (User Space), chương trình sử dụng thiết bị **/dev/mem** kết hợp với hàm **mmap()**. Phương pháp này cho phép ánh xạ các địa chỉ vật lý của hệ thống AXI vào không gian địa chỉ của chương trình mà không cần xây dựng trình điều khiển (Kernel Driver), giúp đơn giản hóa quá trình phát triển và kiểm thử, các vùng địa chỉ được ánh xạ gồm:

```
int denoiser_init(){
    mem_fd=open("/dev/mem",O_RDWR|O_SYNC);
    if(mem_fd<0){perror("open /dev/mem");return -1;}
    ctrl_map =mmap(NULL,MAP_SIZE,PROT_READ|PROT_WRITE,MAP_SHARED,mem_fd,CTRL_BASE);
    bram_in_map =mmap(NULL,MAP_SIZE,PROT_READ|PROT_WRITE,MAP_SHARED,mem_fd,BRAM_IN_BASE);
    bram_out_map=mmap(NULL,MAP_SIZE,PROT_READ|PROT_WRITE,MAP_SHARED,mem_fd,BRAM_OUT_BASE);
    if(ctrl_map==MAP_FAILED||bram_in_map==MAP_FAILED||bram_out_map==MAP_FAILED)
        {perror("mmap");return -1;}
}
```

Hình 5 Ánh xạ bộ nhớ bằng mmap()

- ctrl_map = mmap(0xA000_0000): Ánh xạ vùng s_axi_control.
- bram_in_map = mmap(0xA001_0000): Ánh xạ BRAM đầu vào.
- bram_out_map = mmap(0xA001_2000): Ánh xạ BRAM đầu ra.

3.4.2. Cấu trúc dữ liệu trong BRAM

Do mô hình sử dụng kiểu dữ liệu ap_fixed<16,6>, mỗi giá trị pixel được biểu diễn trên 16 bit. Để tận dụng bus dữ liệu 32 bit của AXI, chương trình đóng gói hai giá trị pixel vào cùng một từ nhớ (Word) 32 bit trước khi ghi xuống BRAM.

Cấu trúc lưu trữ được biểu diễn như sau:

```
/*
 * Ghi 784 pixel vào BRAM_IN
 * ap_fixed<16,6>: scale=1024, pack 2 pixel vào 1 word 32-bit
 * word[i/2] = px[i](bit15:0) | px[i+1](bit31:16)
 */
void load_image(const float *px){
    for(int i=0;i<PIXEL_COUNT;i+=2){
        uint16_t lo=(uint16_t)(int16_t)(px[i ]*FIXED_SCALE);
        uint16_t hi=(uint16_t)(int16_t)(px[i+1]*FIXED_SCALE);
        W(bram_in_map,(i/2)*4, (uint32_t)lo|((uint32_t)hi<<16));
    }
    barrier(); /* đảm bảo BRAM được ghi trước khi IP được */
}
```

Hình 6 Cấu trúc dữ liệu trong BRAM

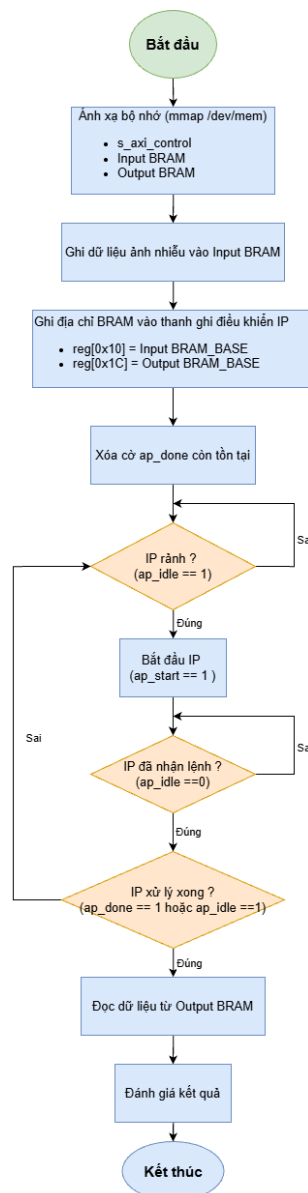
Trong quá trình ghi dữ liệu:

- 784 pixel của một ảnh được chuyển từ dạng số thực sang fixed-point bằng hệ số tỉ lệ 1024.
- Hai pixel liên tiếp được đóng gói vào một Word 32 bit.
- Tổng cộng cần ghi 392 Word vào BRAM đầu vào.

Khi IP Core xử lý hoàn tất, chương trình đọc lại 392 Word từ Output BRAM, tách thành 784 giá trị fixed-point rồi chuyển ngược sang số thực để phục vụ đánh giá chất lượng khử nhiễu.

3.4.3. Luồng điều khiển IP

Sau khi hoàn tất quá trình ánh xạ bộ nhớ, chương trình ARM thực hiện quá trình suy luận của IP Core theo các bước sau:



Hình 7 Lưu đồ điều khiển IP

1. Ghi dữ liệu ảnh nhiễu vào Input BRAM.
2. Ghi địa chỉ của Input BRAM và Output BRAM vào các thanh ghi địa chỉ của IP Core.
3. Đọc thanh ghi điều khiển để xóa cờ **ap_done** còn tồn tại từ lần thực thi trước.
4. Kiểm tra tín hiệu **ap_idle** để bảo đảm IP đang ở trạng thái sẵn sàng.
5. Ghi bit **ap_start** nhằm bắt đầu quá trình suy luận.
6. Theo dõi các tín hiệu **ap_done** và **ap_idle** để xác định thời điểm IP Core hoàn thành xử lý.
7. Đọc dữ liệu từ Output BRAM, giải nén về dạng số thực và lưu kết quả.

Toàn bộ quy trình trên được thực hiện lặp lại cho từng ảnh trong tập dữ liệu kiểm thử. Trong chương trình, các lệnh **barrier()** cũng được sử dụng sau khi ghi hoặc đọc BRAM nhằm bảo đảm thứ tự truy cập bộ nhớ giữa bộ xử lý ARM và IP Core.

3.4.4. Quy trình triển khai trên KV260

Sau khi thiết kế phần cứng hoàn tất, hệ thống được triển khai trên kit KV260 theo các bước:

- Xuất phần cứng từ Vivado dưới dạng bitstream.
- Nạp bitstream xuống FPGA thông qua FPGA Manager trên Linux.
- Biên dịch chương trình điều khiển bằng trình biên dịch GCC.
- Thực hiện chạy chương trình để gửi dữ liệu đến IP Core, nhận kết quả xử lý và lưu phục vụ đánh giá.

Quy trình này giúp kết nối hoàn chỉnh giữa phần mềm chạy trên ARM và phần cứng tăng tốc được hiện thực trong vùng Programmable Logic.

Chương 4. KIỂM TRA VÀ ĐÁNH GIÁ KẾT QUẢ

4.1. Thiết lập môi trường thực nghiệm

Thực hiện trên kit AMD Kria KV260 Vision AI Starter Kit với IP Core được tổng hợp bằng Vitis HLS 2022.2 và tích hợp hệ thống bằng Vivado 2022.2. Phần mềm điều khiển được xây dựng bằng ngôn ngữ C và chạy trên hệ điều hành Ubuntu Linux của KV260.

Để đánh giá mô hình, đề án sử dụng 100 ảnh thuộc tập kiểm thử MNIST. Với mỗi ảnh, ba tập dữ liệu được chuẩn bị gồm:

- `test_input_N.txt`: ảnh nhiễu đưa vào IP Core.

- test_ground_truth_N.txt: ảnh gốc sạch dùng đánh giá chất lượng khử nhiễu.
- gold_output_N.txt: kết quả suy luận của mô hình Python dùng làm chuẩn so sánh với FPGA.

Các chỉ tiêu đánh giá gồm:

- PSNR giữa FPGA và Python (đánh giá độ chính xác khi chuyển đổi sang HLS).
- PSNR giữa FPGA và ảnh gốc (đánh giá chất lượng khử nhiễu).
- Độ trễ xử lý (Latency) và thông lượng của hệ thống (FPS).

4.2. Kết quả C Simulation và CoSim (Vitis HLS)

Trước khi triển khai lên phần cứng, thiết kế được kiểm tra bằng chức năng C Simulation của Vitis HLS nhằm xác minh tính đúng đắn của mã nguồn C++.

Kết quả thử nghiệm trên 100 ảnh cho thấy đầu ra của chương trình C++ có độ tương đồng rất cao với mô hình Python sau huấn luyện

```

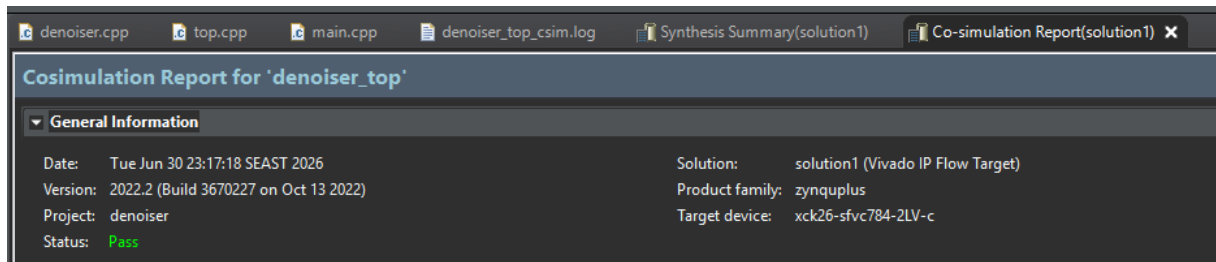
4  Generating CSim.exe
5 Start C++ inference on 100 images...
6 Processed 20 images
7 Processed 40 images
8 Processed 60 images
9 Processed 80 images
10 Processed 100 images
11
12 ===== C++ INFERENCE RESULTS =====
13 Processed images: 100
14
15 --- Accuracy of conversion (C++ vs Python gold) ---
16 Average MSE : 0.0001
17 Std Dev MSE : 0.0000
18 Average PSNR : 43.0038 dB
19 Std Dev PSNR : 1.7294 dB
20
21 --- Denoising quality (C++ vs ground truth) ---
22 Average MSE : 0.0131
23 Std Dev MSE : 0.0045
24 Average PSNR : 19.1179 dB
25 Std Dev PSNR : 1.6387 dB
26

```

Hình 8 Kết quả C Simulation

Sau khi tổng hợp RTL, chức năng **C/RTL CoSimulation** được sử dụng để kiểm tra sự tương đương giữa mô hình C++ và phần cứng Verilog được sinh tự động.

Kết quả CoSimulation cho thấy toàn bộ 100 ảnh đều được xử lý thành công và kết quả RTL hoàn toàn trùng khớp với mô hình C++, chứng minh quá trình tổng hợp HLS không làm thay đổi chức năng của thiết kế.



Hình 9 Kết quả C/RTL CoSimulation

4.3. Kết quả thực nghiệm trên FPGA KV260

Sau khi nạp bitstream và chạy chương trình điều khiển trên ARM, hệ thống được đánh giá trên tập gồm 100 ảnh kiểm thử.

```

BRAM_IN không đ?i IP đ?c nhưng không ghi vào BRAM_IN
root@KV260-01:/home/ubuntu/denoiser#
root@KV260-01:/home/ubuntu/denoiser# gcc -O2 -o denoiser_final denoiser_final.c -lm
root@KV260-01:/home/ubuntu/denoiser# echo "Compile: $?"
Compile: 0
root@KV260-01:/home/ubuntu/denoiser# sudo ./denoiser_final ./hls_test_data 100 2>&1 | tee final_log.txt
[OK] Init: ctrl=0xA0000000 in=0xA0010000 out=0xA0012000

[START] Testing 100 images...
[ 20/100] vs gold: 43.33 dB | vs GT: 19.75 dB | 5011us
[ 40/100] vs gold: 46.46 dB | vs GT: 20.95 dB | 5011us
[ 60/100] vs gold: 42.52 dB | vs GT: 19.48 dB | 5011us
[ 80/100] vs gold: 42.67 dB | vs GT: 18.19 dB | 5011us
[100/100] vs gold: 43.62 dB | vs GT: 20.39 dB | 5011us

-----
K?T QU? FPGA KV260 100 ?nh
-----
FPGA vs Python gold: 42.9725 dB
FPGA vs Ground Truth: 19.1119 dB
Min/Max PSNR vs GT: 16.06 / 23.88 dB
Avg latency: 5.011 ms | 199.6 FPS
Skipped: 0
[OK] fpga_final_results.txt
root@KV260-01:/home/ubuntu/denoiser#
root@KV260-01:/home/ubuntu/denoiser#

```

Hình 10 Kết quả thực nghiệm FPGA KV260 - 100 ảnh

4.4. Đánh giá kết quả

4.4.1. Độ chính xác chuyển đổi Fixed-point

PSNR giữa FPGA output và Python gold đạt 42.97 dB, nghĩa là sai số trung bình chỉ khoảng 0.007 (pixel [0,1]). Đây là mức chính xác rất cao, chứng minh quá trình chuyển đổi từ floating-point sang Fixed-point `ap_fixed<16,6>` và triển khai HLS đã bảo toàn được kết quả tính toán của mô hình gốc.

4.4.2. Chất lượng khử nhiễu

PSNR khử nhiễu đạt 19.11 dB, tương đương với kết quả của C Simulation (19.12 dB) và mô hình Python gốc. Sai lệch không đáng kể (< 0.1 dB) khẳng định việc triển khai phần cứng không làm giảm chất lượng khử nhiễu của mô hình.

4.4.3. Hiệu năng

Thông lượng 199.5 FPS với độ trễ 5.011 ms/ảnh là kết quả đo thực tế bao gồm thời gian ghi/đọc BRAM qua mmap và thời gian xử lý của IP Core. Thời gian này phù hợp với yêu cầu xử lý thời gian thực cho các ứng dụng nhúng.

4.4.4. So sánh giữa C Simulation và FPGA

Bảng 10 So sánh kết quả C Simulation và FPGA thực tế

Chỉ số	C Simulation	FPGA KV260	Sai lệch
PSNR vs gold (dB)	43.00	42.97	0.03 dB
PSNR vs GT (dB)	19.12	19.11	0.01 dB
Kết quả	PASS	PASS	-

Sai lệch cực nhỏ giữa C Simulation và FPGA thực tế xác nhận quy trình HLS synthesis và SoC integration hoàn toàn chính xác. Sai lệch chủ yếu đến từ khác biệt nhỏ trong cách ARM Linux đọc/ghi BRAM so với mô hình lý tưởng trong simulation.

4.4.5. So sánh hiệu năng và tài nguyên với công trình liên quan

Để đánh giá khách quan hiệu quả triển khai phần cứng, đồ án đối chiếu với nghiên cứu của Zhao và cộng sự (2018) - một công trình triển khai Convolutional Autoencoder trên FPGA bằng phương pháp Verilog thủ công (không dùng HLS). Cần lưu ý rằng hai công trình có kiến trúc mô hình và bài toán mục tiêu khác nhau, do đó việc so sánh dưới đây chỉ mang tính chất đối chiếu hiệu quả triển khai phần cứng và quy trình thiết kế, không phải so sánh trực tiếp chất lượng mô hình.

Bảng 11 Đối chiếu hiệu năng và tài nguyên phần cứng với Zhao et al. (2018)

Tiêu chí	Zhao et al. (2018)	Đồ án (KV260)
Nền tảng FPGA	Xilinx KCU105	Xilinx KV260
Phương pháp thiết kế	Verilog thủ công	HLS C++ (Vitis HLS)
Bài toán	Nén ảnh (256×256×3)	Khử nhiễu ảnh (28×28×1)
Số tham số mô hình	68.227	665
LUT	22.441 (9.26%)	~10.059 (~8.6%)
FF	27.100 (5.59%)	~3.475 (~1.5%)
BRAM	55.5 (9.25%)	24 (~16.7%)
Latency / ảnh	15.73 ms	3.95 ms (HLS) / 5.011 ms (thực đo FPGA)
Clock	150 MHz	136.99 MHz

Mặc dù khác biệt về kiến trúc và bài toán, kết quả đối chiếu cho thấy đồ án đạt độ trễ xử lý mỗi ảnh thấp hơn (3.95 ms so với 15.73 ms) với mô hình nhỏ gọn hơn rất nhiều (665 so với 68.227 tham số). Bên cạnh đó, việc sử dụng HLS C++ thay vì thiết kế RTL thủ công bằng Verilog giúp rút ngắn đáng kể thời gian phát triển, cho phép thử nghiệm nhanh nhiều cấu hình tối ưu hóa khác nhau - một lợi thế quan trọng của phương pháp luận HLS so với thiết kế RTL truyền thống.

4.4.6. Tổng kết đánh giá

Kết quả thực nghiệm khẳng định ba điểm chính: (1) việc lựa chọn model nhỏ thay vì model lớn là quyết định hợp lý, đánh đổi 0.93 dB PSNR để giảm 18 lần tham số và đảm bảo khả năng triển khai FPGA; (2) cấu hình pragma tối ưu tài nguyên (INLINE+UNROLL $\times$ 2+Shared Buffer) giúp giảm 83% DSP và 51% FF so với baseline, phù hợp với mục tiêu thiết kế cho thiết bị nhúng; (3) toàn bộ pipeline từ Python đến FPGA thực tế cho kết quả nhất quán (sai lệch PSNR < 0.1 dB giữa C Simulation và FPGA), khẳng định tính đúng đắn và độ tin cậy của quy trình thiết kế.

Chương 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết luận

Đồ án đã thiết kế và hiện thực thành công bộ tăng tốc phần cứng cho mô hình Denoising Autoencoder trên FPGA Xilinx Kria KV260 bằng phương pháp High-Level Synthesis (HLS). Các kết quả chính đạt được gồm:

- Xây dựng và đánh giá hai kiến trúc Autoencoder, từ đó lựa chọn mô hình nhỏ (665 tham số) phù hợp để triển khai trên FPGA.
- Chuyển đổi thành công mô hình từ Python sang C++ HLS sử dụng kiểu dữ liệu `ap_fixed<16,6>`, đạt PSNR khoảng 43 dB so với mô hình Python.
- Áp dụng các kỹ thuật tối ưu HLS như INLINE, UNROLL và Shared Buffer, giúp cân bằng giữa hiệu năng và tài nguyên phần cứng.
- Xây dựng hoàn chỉnh hệ thống SoC trên nền tảng KV260, tích hợp IP Core với bộ xử lý ARM thông qua giao tiếp AXI và phát triển chương trình điều khiển trên Linux.
- Kiểm thử thành công trên phần cứng với PSNR 42,97 dB so với mô hình Python, 19,11 dB so với ảnh gốc và độ trễ trung bình khoảng 5,01 ms/ảnh.

Kết quả đạt được cho thấy phương pháp HLS là hướng tiếp cận hiệu quả để triển khai các mô hình học sâu kích thước nhỏ trên FPGA, đáp ứng tốt yêu cầu về độ chính xác và tài nguyên của các hệ thống nhúng.

5.2. Hướng phát triển

Trong tương lai, đề án có thể được mở rộng theo một số hướng sau:

- Tối ưu kiến trúc HLS: Nghiên cứu áp dụng kỹ thuật DATAFLOW nhằm tăng khả năng thực thi song song giữa các lớp, từ đó giảm độ trễ của toàn hệ thống.
- Tối ưu độ rộng dữ liệu: Khảo sát các định dạng số có độ rộng nhỏ hơn như `ap_fixed<8,4>` hoặc INT8 để giảm thêm tài nguyên BRAM và DSP, đồng thời đánh giá mức ảnh hưởng đến chất lượng xử lý.
- Mở rộng mô hình: Triển khai các kiến trúc sâu hơn như U-Net hoặc các Autoencoder có số lượng tham số lớn hơn kết hợp với DDR và AXI DMA để xử lý các bài toán ảnh có kích thước lớn.
- Xử lý thời gian thực: Kết nối trực tiếp camera với kit KV260 để xây dựng hệ thống xử lý ảnh hoặc video theo thời gian thực trên nền FPGA.
- Tự động hóa quy trình triển khai: Phát triển công cụ tự động từ quá trình huấn luyện mô hình, chuyển đổi trọng số, sinh mã C++ HLS, tổng hợp IP Core, tích hợp Vivado đến kiểm thử trên FPGA nhằm giảm thời gian phát triển và thuận tiện cho việc triển khai các mô hình khác.

TÀI LIỆU THAM KHẢO

- [1] Vincent, P., Larochelle, H., Bengio, Y., & Manzagol, P.-A. (2008). Extracting and Composing Robust Features with Denoising Autoencoders. Proceedings of the 25th International Conference on Machine Learning (ICML), 1096-1103.
- [2] Vincent, P., Larochelle, H., Lajoie, I., Bengio, Y., & Manzagol, P.-A. (2010). Stacked Denoising Autoencoders: Learning Useful Representations in a Deep Network with a Local Denoising Criterion. Journal of Machine Learning Research, 11, 3371-3408.
- [3] Trần Tùng Nhi, Phạm Văn Quân, & Phan Công Đạt (2022). Ứng dụng kỹ thuật bộ mã tự động tích chập (Convolutional Autoencoder) trong xử lý ảnh. Tạp chí Khoa học Đại học Đông Á, Số 01, 75-80.
- [4] Zhao, W., Jia, Z., Wei, X., & Wang, H. (2018). An FPGA Implementation of a Convolutional Auto-Encoder. Applied Sciences, 8(4), 504.
- [5] LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-Based Learning Applied to Document Recognition. Proceedings of the IEEE, 86(11), 2278-2324.
- [6] Xilinx (AMD). (2022). Vitis HLS User Guide (UG1399). AMD/Xilinx Documentation.
- [7] Solovyev, R., et al. (2018). Fixed-Point Convolutional Neural Network for Real-Time Video Processing in FPGA. arXiv:1808.09945.
- [8] Xilinx (AMD). (2023). AXI Reference Guide (UG1037). AMD/Xilinx Documentation.
- [9] LeCun, Y., Cortes, C., & Burges, C. J. C. (1998). The MNIST Database of Handwritten Digits. <http://yann.lecun.com/exdb/mnist/>
- [10] Xilinx (AMD). (2022). Kria KV260 Vision AI Starter Kit User Guide (UG1089).
- [11] Nair, V., & Hinton, G. E. (2010). Rectified Linear Units Improve Restricted Boltzmann Machines. ICML-10 Proceedings.